

Jugando al BlackJack: Simulación de Blackjack para Aprendizaje por Refuerzo

28 de mayo de 2026

Expediente	Nombre
223201410	Zurita Murillo Juan Pablo

■ Descripción del Entorno

El objetivo es modelar el juego de Blackjack simplificado como un Proceso de Decisión de Markov (MDP) pero para ser simulado, lo que lo hace más simple.

Reglas Base:

- Baraja: Infinita con reemplazo
- Valores: Las figuras (J, Q, K) valen 10. El As vale 1 u 11 (es “usable” si puede valer 11 sin que el jugador se pase de 21).
- El Crupier: política fija, pide carta (hit) si su suma es menor a 17 y se planta (stand) si es 17 o más.

■ Definición de los Componentes del MDP

- Espacio de Estados (S)

$s = (\text{Suma Jugador}, \text{Carta Visible Crupier}, \text{As Usable})$

- Suma del Jugador: Entero entre 12 y 21. Si es < 12 , el jugador siempre debe pedir carta.
- Carta Visible del Crupier: Entero del 1 al 10 (donde 1 representa al As).
- As Usable: Booleano (Verdadero/Falso).

¿Cuál es la cardinalidad del espacio de estado?

$$|S| = 10 \times 10 \times 2 = 200$$

- Espacio de Acciones (A)

En cada estado, el agente puede elegir entre dos acciones discretas:

- Plantarse (0 - Stand): El jugador termina su turno y se compara su suma con la del crupier.

- Pedir (1 - Hit): El jugador recibe una carta adicional.

¿Cómo se puede encontrar las acciones legales en cada estado?

Las acciones legales son las mismas para todo estado:

$$A(s) = \{0, 1\}$$

Plantarse o pedir.

- Dinámica y Estados Terminales

El juego termina cuando:

1. El jugador se planta.
2. El jugador pide y su suma > 21 .
3. Blackjack natural.

- Función de Recompensa (R)

La recompensa en cada estado no terminal es 0. Se otorga únicamente al final del episodio (estado terminal) un valor como sigue:

- +1: El jugador gana
- 0: Empate (Push).
- -1: El jugador pierde o se pasa (Bust).
- +1.5: Victoria por Blackjack natural (21 con las dos primeras cartas).

- A desarrollar

Propón como hacer los siguientes métodos:

- estado_inicial(self)

- Repartir dos cartas a jugador y crupier
- Checamos si hay un as usable.
- Checamos si hay blackjack natural.
- Si la suma del jugador no da 12 o más, seguimos repartiéndole al jugador.
- Retornamos estado inicial con la suma del jugador, la carta visible del crupier y el booleano del as usable.

- acciones_legales(self, s)

Podemos pedir o plantarnos siempre que no estemos en el estado terminal.

- sucesor(self, s, a)

- Checamos si estamos en estado terminal. De ser cierto regresamos estado terminal.
- Si la acción a fue pedir, agregamos una carta a las cartas del jugador y evaluamos la mano.
 - ◊ Si se pasa de 21 se acaba el juego, si no pasamos al siguiente estado con la estructura (suma del jugador, carta visible del crupier, as visible).
- Si la acción fue plantarse, evaluamos la mano del crupier. Cómo describe el problema, el crupier pide hasta llegar a un número mayor o igual a 17. Si se añadieron cartas, evaluamos la mano de nuevo y regresamos el estado terminal.

- `recompensa(self, s, a, s_p)`
 - Checamos si el siguiente estado es terminal y el actual no lo es.
 - ◊ Si eso es cierto, comparamos la suma de la mano del jugador y la del crupier y premiamos o castigamos según los puntajes descritos anteriormente.
 - ◊ Si no es cierto regresamos 0, ya que la recompensa en cualquier estado no terminal es 0.
- `reparte_carta()`

Tendremos una variable global `BARAJA` con los siguientes elementos:

```
BARAJA = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 10, 10, 10]
```

La función regresa con **choice** (de la biblioteca **random**) una de las cartas.